

Übersicht

Begriff	Beschreibung	Technische Details	Besondere Merkmale
Abstraktion	Konzept zur Vereinfachung komplexer Systeme durch Herausfiltern wesentlicher Eigenschaften	Trennung von Schnittstelle und Implementierungsdetails; Definiert ein Modell bzw. eine vereinfachte Repräsentation der Realität, ohne sich auf konkrete Details festzulegen; Grundlage für Interfaces, abstrakte Klassen und Design-Patterns in objektorientierten Systemen	Erlaubt die Fokussierung auf relevante Aspekte, fördert Wiederverwendbarkeit und Modularität
Objekt	Instanz einer Klasse, die einen konkreten Zustand und ein definiertes Verhalten besitzt	Speicherallokation erfolgt zur Laufzeit; Verfügt über eigene Zustandsvariablen (Attribute) und Methoden, die das Verhalten beschreiben; Lebenszyklus wird durch Konstruktion, Nutzung und Zerstörung (Garbage Collection) gesteuert; Zustandsänderungen erfolgen durch Methodenaufrufe oder interne Logik	Realisierung der Entitäten des Domänenmodells; dynamisch erzeugt und zur Laufzeit veränderbar
Klasse	Blaupause oder Vorlage zur Erzeugung von Objekten, definiert Attribute und Methoden	Statische Struktur, die im Kompilierungsprozess festgelegt wird; enthält Typinformationen, Zugriffsmodifikatoren, Erbstruktur (Vererbungshierarchie) und implementierte Schnittstellen; dient als zentraler Baustein für die Modellierung von Daten und Verhalten in objektorientierten Sprachen	Dient der Wiederverwendung und der logischen Trennung von Verantwortlichkeiten; bildet den Kern der objektorientierten Programmierung
Klassensignatur	Deklaration einer Klasse, die den Namen, Zugriffsmodifikatoren, Vererbungshierarchie und Schnittstellen definiert	Besteht aus Modifikatoren (z. B. public, abstract, final), dem Klassennamen, optionalen Typparametern, der Deklaration der Oberklasse und der Liste implementierter Interfaces; Definiert den Vertrag, der von allen Instanzen der Klasse eingehalten werden muss	Dient der Festlegung der Schnittstelle und Struktur, ohne Details der Implementierung zu offenbaren
Kapselung	Prinzip, das den Zugriff auf interne Zustände einer Klasse beschränkt und kontrolliert	Umsetzung durch Zugriffsmodifikatoren (private, protected, public); ermöglicht die Trennung von Schnittstellendeklaration und interner Implementierung; unterstützt die Kontrolle und Validierung von Zustandsänderungen über definierte Methoden (Getter/Setter); verhindert direkten Zugriff auf interne Datenstrukturen	Fördert Sicherheit, Wartbarkeit und Flexibilität; reduziert Kopplung zwischen Komponenten

Begriff	Beschreibung	Technische Details	Besondere Merkmale
Attribut	Variable oder Eigenschaft, die den Zustand einer Klasse bzw. eines Objekts beschreibt	Wird in der Klassendefinition deklariert und durch einen Datentyp und einen Bezeichner charakterisiert; kann Zugriffsmodifikatoren besitzen; definiert Speicherplatz im Objekt (bei Instanzattributen) oder in der Klassenstruktur (bei statischen Attributen); bildet die Datenbasis der Objektzustände	Zentraler Bestandteil zur Beschreibung des internen Zustands; kann mutable oder immutable sein, je nach Implementierung und Zugriffsregeln
Datentypen	Definitionen, die den erlaubten Wertebereich und Operationen auf Werten festlegen	Unterscheidung in primitive Datentypen (z. B. int, boolean) und Referenztypen (z. B. Klassen, Interfaces); bestimmen Speicherbedarf, Rechenpräzision, und erlaubte Operationen; statische Typprüfung zur Kompilierzeit; fundamental für die Speichermanagement- und Optimierungsmechanismen der Laufzeitumgebung	Bilden die Basis für Typsicherheit und erlauben Optimierungen im Compiler; gewährleisten, dass nur kompatible Operationen auf Werten durchgeführt werden
Attributsignatur	Deklaration eines Attributs bestehend aus Zugriffsmodifikatoren, Datentyp und Bezeichner	Enthält explizite Typangabe, Zugriffsrechte und kann zusätzliche Modifikatoren (final, static) aufweisen; bildet den vertraglichen Bestandteil der Klassenstruktur, der zur Dokumentation und Kompilierzeitüberprüfung dient; definiert, welche Art von Daten gespeichert werden können	Dient der formalen Definition des Zustands einer Klasse; ist Grundlage für die automatische Dokumentation und Reflektion über den Code
Getter	Methode zur kontrollierten Auslesung des Wertes eines Attributs	Stellt eine Schnittstelle zur Verfügung, die den aktuellen Zustand eines Attributs zurückgibt; typischerweise ohne Seiteneffekte; Rückgabotyp entspricht dem Datentyp des Attributs; Teil der kontrollierten Zugriffskontrolle in Kapselungskonzepten; definiert durch Namenskonventionen (z. B. getX)	Erlaubt den schreibgeschützten Zugriff auf private Daten; fördert die Trennung zwischen interner Repräsentation und externer Schnittstelle
Setter	Methode zur kontrollierten Änderung des Wertes eines Attributs	Bietet eine definierte Schnittstelle zum Setzen bzw. Aktualisieren des internen Zustands; nimmt einen Parameter entgegen, der dem Datentyp des Attributs entspricht; ermöglicht Validierung oder Transformationen vor der Zuweisung; definiert durch Namenskonventionen (z. B. setX)	Sichert Datenintegrität durch Validierungslogik; kapselt die Zustandsänderung, um unerwünschte direkte Modifikationen zu verhindern
final Attribut	Attribut, dessen Wert nach der Initialisierung nicht mehr verändert werden kann	Deklariert mit dem Modifier <code>final</code> ; muss entweder bei Deklaration oder im Konstruktor initialisiert werden; gewährleistet, dass die Referenz bzw. der Wert unveränderlich ist; Teil der Immutabilitätsstrategie in der objektorientierten Programmierung	Erhöht die Sicherheit und Vorhersagbarkeit des Codes; oft in Kombination mit <code>static</code> zur Definition von Konstanten

Begriff	Beschreibung	Technische Details	Besondere Merkmale
static Attribut	Attribut, das zur Klassenebene gehört und für alle Instanzen der Klasse gemeinsam existiert	Deklariert mit dem Modifier <code>static</code> ; einmalige Initialisierung beim Laden der Klasse; Speicherort im statischen Bereich der Laufzeitumgebung; ermöglicht globalen, klassenübergreifenden Zugriff innerhalb des Anwendungsbereichs; häufig in Verbindung mit <code>final</code> für unveränderliche Konstanten	Erlaubt den Zugriff ohne Instanziierung; zentral für die Verwaltung gemeinsamer Daten und Ressourcen auf Klassenebene
final Methode	Methode, die nicht durch Unterklassen überschrieben werden kann	Deklariert mit dem Modifier <code>final</code> im Methodenkopf; stellt sicher, dass die Implementierung in der Basisklasse erhalten bleibt; fördert die Vorhersagbarkeit von Verhalten in einer Klassenhierarchie; kann in Kombination mit anderen Modifikatoren (z. B. <code>public</code>) verwendet werden	Sichert kritische Logik gegen unbeabsichtigte oder absichtliche Modifikationsversuche in Unterklassen
static Methode	Methode, die zur Klassenebene gehört und ohne Instanzaufruf ausgeführt werden kann	Deklariert mit dem Modifier <code>static</code> ; kann ohne ein Objekt der Klasse aufgerufen werden; operiert in einem statischen Kontext, weshalb auf Instanzvariablen nicht direkt zugegriffen werden kann; wird typischerweise für Hilfsfunktionen oder Utility-Methoden genutzt, die keinen Objektzustand benötigen	Ermöglicht den Aufruf von Methoden, ohne vorherige Objekterstellung; unterstützt das Design von service-orientierten oder utility-orientierten Klassenstrukturen